Lab-18.3

Name: D. Suraj
2303A53006
Batch:46

Lab 18 – API Integration: Connecting to External Services with Error Handling

Task 1 – Public Transport Route Fare API

Task:
Integrate a Public Transport API to retrieve fare details between two stations.
Instructions:
- Use AI to generate API request code
- Validate source and destination inputs
- Handle API downtime and invalid station names
- Display fare details in a structured format

Expected Output:
A working script that retrieves transport fare information with graceful error handling.

Prompt:
"Generate a Python script to fetch public transport fare details between two stations using a REST API. Include input validation, handle 404 errors for invalid stations, and display output in a structured format."
Code:

```python
import requests
def get_london_transport_info():
    # Using a real, live Public Transport API (TfL)
    # This fetches the status of a specific transport line (e.g., 'victoria' or 'central')
    line_id = input("Enter London Underground Line (e.g., victoria, central, jubilee): ").strip().lower()
    url = f"https://api.tfl.gov.uk/Line/{line_id}/Status"
    try:
        print(f"\nFetching status for {line_id} line...")
        response = requests.get(url, timeout=10)
        # Check if the line name was valid (404 check)
        if response.status_code == 404:
            print(f"Error: The line '{line_id}' does not exist.")
            return
        response.raise_for_status()
        data = response.json()
        # The API returns a list; we take the first element
        status_details = data[0]['lineStatuses'][0]
        status_name = status_details['statusSeverityDescription']
        reason = status_details.get('reason', 'No delays reported.')
        # Structured Output for Lab Report
        print("="*40)
        print(f"TRANSPORT LINE: {line_id.upper()}")
        print(f"CURRENT STATUS: {status_name}")
        print(f"REASON        : {reason}")
        print("="*40)
    except requests.exceptions.ConnectionError:
        print("Error: Could not connect to the Transport API. Check your internet.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
if __name__ == "__main__":
    get_london_transport_info()
```

Output:

```
PS C:\Users\dussa\OneDrive\Desktop\Devops project> & "c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe" "c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle
d\libs\debugpy\launcher" "59500" "--" "c:\Users\dussa\OneDrive\Desktop\Traning\ai.py"
Enter London Underground Line (e.g., victoria, central, jubilee): victoria

Fetching status for victoria line...
========================================
Enter London Underground Line (e.g., victoria, central, jubilee): victoria

Fetching status for victoria line...
========================================

Fetching status for victoria line...
========================================
Fetching status for victoria line...
========================================
========================================
TRANSPORT LINE: VICTORIA
CURRENT STATUS: Good Service
REASON        : No delays reported.
REASON        : No delays reported.
========================================
PS C:\Users\dussa\OneDrive\Desktop\Devops project> ^C
```

Task 2 – Currency Exchange Rates

Task:
Integrate a Currency Exchange API to convert INR to USD, EUR, and GBP.
Instructions:
• Use AI to generate API request code.
• Validate user input for amount and currency codes.
• Implement error handling for invalid responses and API downtime.
• Show conversion results clearly in tabular format.
Expected Output:
• A script that converts INR to multiple currencies with graceful error handling.
Prompt:
"Write a Python script to convert INR to USD, EUR, and GBP using the ExchangeRate-API. Validate that the user enters a valid number. Display the results in a clean tabular format using f-string alignment."
Code:

```python
import requests
def convert_currency():
    # Using the public ExchangeRate-API (No key required for basic V4)
    url = "https://api.exchangerate-api.com/v4/latest/INR"
    print("--- Real-Time Currency Converter (INR) ---")
    try:
        # 1. Input Validation
        amount_str = input("Enter amount in INR: ").strip()
        if not amount_str:
            print("Error: Amount cannot be empty.")
            return
        inr_amount = float(amount_str)
        if inr_amount <= 0:
            print("Error: Please enter a positive amount.")
            return
        # 2. API Request
        print("Fetching latest exchange rates...")
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        data = response.json()
        rates = data.get("rates", {})
        targets = ["USD", "EUR", "GBP"]
        # 3. Tabular Output
        print("\n" + "="*45)
        print(f"{'CURRENCY':<12} | {'RATE':<12} | {'CONVERTED'}")
        print("-" * 45)
        for code in targets:
            rate = rates.get(code)
            if rate:
                converted = inr_amount * rate
                # Formatting: <12 for alignment, .4f for rate precision, .2f for money
                print(f"{code:<12} | {rate:<12.4f} | {converted:>10.2f}")
        print("="*45)
        print(f"Data last updated: {data.get('date')}")
    except ValueError:
        print("Error: Invalid input. Please enter a numeric value (e.g., 500.50).")
    except requests.exceptions.ConnectionError:
        print("Error: Could not connect to the exchange rate service.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
if __name__ == "__main__":
    convert_currency()
```

Output:

```
PS C:\Users\dussa\OneDrive\Desktop\Deovps project> & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '60563' '--' 'c:\Users\dussa\OneDrive\Desktop\Traning\ai.py'
--- Real-Time Currency Converter (INR) ---
Enter amount in INR: 1000
Fetching latest exchange rates...

=============================================
CURRENCY     | RATE         | CONVERTED
---------------------------------------------
USD          | 0.0106       |      10.60
EUR          | 0.0092       |       9.19
GBP          | 0.0080       |       7.95
=============================================
Data last updated: 2026-03-25
PS C:\Users\dussa\OneDrive\Desktop\Deovps project>
```

Explanation :

- **API Usage:** Utilized api.exchangerate-api.com to fetch live JSON data.
- **Data Parsing:** Accessed the rates dictionary to extract specific multipliers for USD, EUR, and GBP.
- **Formatting:** Employed f-string padding (:<12) to ensure the table columns are vertically aligned regardless of the number length.
- **Error Handling:** Added a ValueError check to prevent the program from crashing if a user types text instead of numbers.

Task 3 – GitHub Repository Info Fetcher

Task:
Connect to the GitHub API to fetch details of a repository (e.g., stars, forks, issues).
Instructions:
• Prompt AI to write a Python function to send GET requests to GitHub API.
• Handle rate limit errors, 404 (repo not found), and invalid input.
• Display repository name, description, stars, forks, and open issues.
Expected Output:
• A Python program that retrieves and displays GitHub repository information with robust error handling.

Prompt:
"Write a Python function to fetch details of a GitHub repository (stars, forks, open issues) using the GitHub API. Handle 404 errors for missing repositories and 403 errors for rate limits. Display the name, description, and stats clearly."
Code:

```python
import requests
def fetch_github_info():
    print("--- GitHub Repository Statistics Fetcher ---")
    # Example Input: pallets/flask or tensorflow/tensorflow
    repo_path = input("Enter 'owner/repository' (e.g., pallets/flask): ").strip()

    if "/" not in repo_path:
        print("Error: Invalid format. Please use 'owner/repo_name'.")
        return

    url = f"https://api.github.com/repos/{repo_path}"

    try:
        print(f"Connecting to GitHub API for {repo_path}...")
        response = requests.get(url, timeout=10)

        # Specific GitHub Status Code Handling
        if response.status_code == 404:
            print(f"Error: The repository '{repo_path}' was not found.")
            return
        elif response.status_code == 403:
            print("Error: API Rate limit exceeded. Please wait a few minutes.")
            return
        response.raise_for_status()
        data = response.json()
        # Displaying meaningful information
        print("\n" + "="*40)
        print(f"REPO NAME   : {data.get('full_name')}")
        print(f"DESCRIPTION : {data.get('description')}")
        print("-" * 40)
        print(f"⭐ STARS     : {data.get('stargazers_count'):,}")
        print(f"🍴 FORKS     : {data.get('forks_count'):,}")
        print(f"❗ ISSUES    : {data.get('open_issues_count'):,}")
        print("="*40)

    except requests.exceptions.ConnectionError:
        print("Error: Could not connect to GitHub. Check your internet.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")

if __name__ == "__main__":
    fetch_github_info()
```

Output:

```
PS C:\Users\dussa\OneDrive\Desktop\Devops project>
--- GitHub Repository Statistics Fetcher ---
Enter 'owner/repository' (e.g., pallets/flask): pallets/flask
Connecting to GitHub API for pallets/flask...

========================================
REPO NAME   : pallets/flask
DESCRIPTION : The Python micro framework for building web applications.
----------------------------------------
⭐ STARS     : 71,379
🍴 FORKS     : 16,758
❗ ISSUES    : 3
========================================
PS C:\Users\dussa\OneDrive\Desktop\Devops project>
```

Explanation:

- **REST API Integration:** Used the GET method to request data from GitHub's repository endpoint.
- **Status Code Logic:** Explicitly checked for **404 (Not Found)** to handle typos in repository names and **403 (Forbidden)** to handle GitHub's secondary rate limits.
- **Data Presentation:** Used the :, formatting in f-strings to ensure large numbers (like stars) are displayed with thousands-separators for better readability.
- **Resilience:** The script includes a 10-second timeout to prevent the program from hanging if GitHub's servers are slow.

Task 4 – Real-Time Application: News Headlines Aggregator

Scenario:
Build a News Aggregator using a free News API.
Requirements:
• Fetch top 5 headlines for a given category (sports, technology, health).
• Handle errors like invalid category, missing API key, and request failures.
• Display headlines in a user-friendly numbered list format.
• Include a retry mechanism if the first request fails.
Expected Output:
• A working Python script that retrieves and displays news headlines with proper error handling
Prompt: "Write a Python script to fetch the top 5 news headlines for a specific category (e.g., technology, sports, business). Implement a retry mechanism that attempts the request up to 3 times if it fails. Display the headlines in a numbered list."

Code:

```python
import requests
import time
def fetch_live_news():
    # Using a Public RSS-to-JSON feed of Google News (No API Key Required)
    # This is a reliable way to get LIVE news for a lab project.
    category = input("Enter news category (technology, sports, business): ").strip().lower()
    # Mapping user category to a live RSS feed
    rss_feeds = {
        "technology": "https://news.google.com/rss/search?q=technology&hl=en-IN&gl=IN&ceid=IN:en",
        "sports": "https://news.google.com/rss/search?q=sports&hl=en-IN&gl=IN&ceid=IN:en",
        "business": "https://news.google.com/rss/search?q=business&hl=en-IN&gl=IN&ceid=IN:en"
    }
    target_url = rss_feeds.get(category, rss_feeds["technology"])
    # We use a public 'rss2json' converter to turn the feed into a readable API response
    api_url = f"https://api.rss2json.com/v1/api.json?rss_url={target_url}"
    max_retries = 3
    for attempt in range(max_retries):
        try:
            print(f"Connecting to Live News Stream (Attempt {attempt + 1})...")
            response = requests.get(api_url, timeout=10)
            response.raise_for_status()
            data = response.json()
            articles = data.get('items', [])[:5] # Get top 5 live items
            if not articles:
                print("No live news found for this category.")
                return
            print(f"\n--- LIVE {category.upper()} HEADLINES (REAL-TIME) ---")
            for i, article in enumerate(articles, 1):
                # Clean up the title (Google News titles often include the source)
                title = article.get('title', 'No Title')
                pub_date = article.get('pubDate', 'Recently')
                print(f"{i}. {title}")
                print(f"   Published: {pub_date}")
            break # Exit on success
        except Exception as e:
            print(f"Request failed: {e}")
            if attempt < max_retries - 1:
                print("Waiting 2 seconds before retrying...")
                time.sleep(2)
            else:
                print("Max retries reached. Check your internet connection.")
if __name__ == "__main__":
    fetch_live_news()
```

Output:

```
PS C:\Users\dussa\OneDrive\Desktop\Deovps project>  & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '60
954' '--' 'c:\Users\dussa\OneDrive\Desktop\Traning\ai.py'
Enter news category (technology, sports, business): technology
Connecting to Live News Stream (Attempt 1)...

--- LIVE TECHNOLOGY HEADLINES (REAL-TIME) ---
1. Social media bans and digital curfews to be trialled on UK teenagers - BBC
   Published: 2026-03-24 22:37:31
2. World first as development of quantum navigation technology advances onto Britain's national railway - Network Rail media centre
   Published: 2026-03-24 14:07:41
3. Divide between Silicon Valley and ordinary people grows ever larger | Technology - The Guardian
   Published: 2026-03-24 14:06:00
4. Carbon Capture Technology Is Helping This Pub Make Beer - The New York Times
   Published: 2026-03-23 15:30:46
5. German Nuclear Fusion Startup Plans Lasers for Defense, Health - Bloomberg.com
   Published: 2026-03-25 05:30:00
PS C:\Users\dussa\OneDrive\Desktop\Deovps project>
```

Explanation:

- **RSS-to-JSON Integration:** Instead of a restricted REST API, this script utilizes a public converter to process live RSS feeds, demonstrating flexibility in data sourcing.
- **Dynamic URL Construction:** The script builds the API request URL based on user input, mapped through a local dictionary.
- **Real-time Data Handling:** The output includes timestamps (pubDate), showing the ability to handle and display temporal metadata.
- **Resiliency:** The retry mechanism ensures that the program can handle temporary network timeouts typical of free public endpoints.

Task 5 -COVID-19 Statistics API Integration

Task:
Connect to a COVID-19 Statistics API to fetch country-wise COVID data.

Instructions:

- Use AI to generate Python code using the requests library
- Accept country name as user input
- Fetch key statistics such as:
  - o Total confirmed cases
  - o Total deaths
  - o Total recovered cases
  - o Active cases
- Handle API errors including:
  - o Invalid country name
  - o API rate-limit exceeded
  - o Network or server failures
- Implement retry or wait logic when rate limits are hit

Expected Output:
A Python program that retrieves and displays country-wise COVID-19 statistics with proper rate-limit handling and error management.

Prompt:

"Write a Python script to fetch real-time COVID-19 statistics for a specific country using the Disease.sh API. Handle invalid country names with a 404 check and format large numbers with commas for better readability."

Code:

C: > Users > dussa > OneDrive > Desktop > Traning > ai.py > time_stamp_format

```python
import requests
def get_covid_stats():
    print("--- Real-Time COVID-19 Country Tracker ---")
    country_input = input("Enter country name (e.g., India, USA, Japan): ").strip()
    if not country_input:
        print("Error: Please enter a country name.")
        return
    # Official open-access endpoint (No API Key needed)
    url = f"https://disease.sh/v3/covid-19/countries/{country_input}"
    try:
        print(f"Fetching latest data for {country_input}...")
        response = requests.get(url, timeout=10)
        # 1. Handle Invalid Country (404)
        if response.status_code == 404:
            print(f"Error: Statistics for '{country_input}' not found. Check the spelling.")
            return
        # 2. Handle Rate Limits (429)
        if response.status_code == 429:
            print("Error: Too many requests. Please wait a moment.")
            return
        response.raise_for_status()
        data = response.json()
        # 3. Structured and Formatted Output
        # Using {:,} to add commas to large numbers automatically
        print("\n" + "="*40)
        print(f"LOCATION    : {data.get('country').upper()}")
        print(f"POPULATION  : {data.get('population'):,}")
        print("-" * 40)
        print(f"TOTAL CASES : {data.get('cases'):,}")
        print(f"TOTAL DEATHS: {data.get('deaths'):,}")
        print(f"RECOVERED   : {data.get('recovered'):,}")
        print(f"ACTIVE      : {data.get('active'):,}")
        print("="*40)
        print(f"Last Updated: {time_stamp_format(data.get('updated'))}")
    except requests.exceptions.ConnectionError:
        print("Error: Could not connect to the health database.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
def time_stamp_format(ms):
    # Helper to convert API timestamp (ms) to a readable string
    from datetime import datetime
    return datetime.fromtimestamp(ms/1000).strftime('%Y-%m-%d %H:%M:%S')
if __name__ == "__main__":
    get_covid_stats()
```

Output:

```
PS C:\Users\dussa\OneDrive\Desktop\Deovps project>  & "c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe" "c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher" "50
407" "--" "c:\Users\dussa\OneDrive\Desktop\Traning\a1.py"
--- Real-Time COVID-19 Country Tracker ---
Enter country name (e.g., India, USA, Japan): India
Fetching latest data for India...

========================================
LOCATION    : INDIA
POPULATION  : 1,406,631,776
----------------------------------------
TOTAL CASES : 45,035,393
TOTAL DEATHS: 533,570
RECOVERED   : 0
ACTIVE      : 44,501,823
========================================
Last Updated: 2026-03-25 14:01:21
PS C:\Users\dussa\OneDrive\Desktop\Deovps project>
```

Explanation:

- **API Endpoint: Used disease.sh, an industry-standard open API that provides data sourced from Worldometers and Johns Hopkins University.**
- **Thousand Separators: Implemented :, in f-strings. This is a critical feature for statistical apps as it makes large numbers like 1408044253 readable as 1,408,044,253.**
- **Timestamp Conversion: The API provides time in milliseconds. The script uses the datetime module to convert this into a human-readable format.**
- **Error Prevention: The script includes checks for empty strings and specific HTTP status codes (404 and 429) to ensure a robust user experience.**